

UmweltMonitor3000

Technische Dokumentation

Plattform: Windows 11 | .NET 8 | C# / WPF
Entwickler: Dazooo
Version: 1.0 | 2026

1. Projektübersicht

UmweltMonitor3000 ist eine WPF-Desktop-Anwendung auf Basis von .NET 8 und C#. Sie dient zur Überwachung von Pflanzenfeuchtigkeit und Umweltdaten in Echtzeit über das MQTT-Protokoll. Sensordaten werden empfangen, gespeichert und visuell aufbereitet angezeigt.

Die Anwendung ermöglicht die gleichzeitige Überwachung mehrerer Pflanzen mit Feuchtigkeitssensoren und gibt auf Basis von Schwellenwerten automatische Handlungsempfehlungen.

1.1 Technologie-Stack

Technologie	Beschreibung
.NET 8 / C#	Hauptplattform und Programmiersprache
WPF	Windows Presentation Foundation – UI-Framework
CommunityToolkit.Mvvm	MVVM-Framework (ObservableObject, RelayCommand)
LiveChartsCore	Diagramm-Bibliothek (Linien-, Balken-, PieCharts)
MaterialDesignThemes	UI-Design-Bibliothek (Dark Theme, Icons)
MQTTnet	MQTT-Client-Bibliothek
SQLite	Lokale Datenbank fuer historische Sensordaten
System.Text.Json	Persistenz fuer Pflanzen- und Log-Daten

2. Projektstruktur

Die Anwendung folgt einer klaren Schichtentrennung nach dem MVVM-Muster:

```
UmweltMonitor3000/  
% % % App.xaml / App.xaml.cs  
% % % Application/  
%   % % % Interfaces/  
%   %   % % % IMqttService.cs  
%   %   % % % IUmweltMonitorRepository.cs  
%   %   % % % INotificationService.cs  
%   % % % Models/  
%   %   % % % Plant.cs / SensorData.cs / TopicLog.cs  
%   % % % Repositories/  
%   %   % % % UmweltMonitorRepository.cs  
%   % % % Services/  
%   %   % % % MqttService.cs / NotificationService.cs / DarkTitleBar.cs  
%   % % % ViewModels/  
%   %   % % % MainViewModel.cs / MainWindowLogic.cs  
%   %   % % % MqttViewModel.cs / PlantViewModel.cs / StatisticViewModel.cs  
% % % Views/  
% % % % MainWindow.xaml / MainWindow.xaml.cs  
% % % % Converter/  
% % % % % BoolToColor.cs / BoolToIconConverter.cs  
% % % % % ValueToRangeConverter.cs / ValueToRangeConverter2.cs
```

3. Architektur (MVVM)

Die Anwendung verwendet das MVVM-Muster (Model-View-ViewModel), das eine klare Trennung zwischen UI und Logik gewährleistet.

3.1 Übersicht der ViewModels

ViewModel	Verantwortlichkeit
MainViewModel	Root-ViewModel; enthält MqttViewModel, PlantViewModel, StatisticViewModel
MainWindowLogic	Vermittler zwischen Services und ViewModels; Events: LogMessage, ErrorLogged, MessageLogged
MqttViewModel	MQTT-Verbindungssteuerung, Log-Anzeige, UpTime-Timer
PlantViewModel	Pflanzenverwaltung, Sensor-Payload-Parsing, JSON-Persistenz
StatisticViewModel	Chart-Daten, Handlungsempfehlung, Feuchtigkeits- und Temperaturanzeige

3.2 Kommunikation zwischen Schichten

Die MainWindowLogic stellt drei Events bereit, die von allen ViewModels abonniert werden:

- LogMessage – allgemeine Statusmeldungen (z.B. Verbindung hergestellt)
- ErrorLogged – Fehlermeldungen (erhöhen den Fehlerzähler im MqttViewModel)
- MessageLogged – empfangene MQTT-Nachrichten mit Topic und Payload

Alle ViewModels aktualisieren ihre Eigenschaften auf dem UI-Thread via Dispatcher.Invoke.

4. Datenmodell

4.1 Plant

Eigenschaft	Typ	Beschreibung
PlantID	int	Eindeutige ID der Pflanze
Name	string	Name der Pflanze (= letztes MQTT-Topic-Segment)
MqttTopic	string (readonly)	umweltmonitor/sensor/{Name}
MoistureSensor	int	Rohwert des Feuchtigkeitssensors
MoisturePercent	int	Feuchtigkeit in Prozent (0-100)
BatteryState	int	Batteriestand in Prozent (0-100)
LastUpdate	DateTime	Zeitstempel der letzten Sensoraktualisierung

4.2 SensorData

Historische Messwerte in SQLite:

- Topic – MQTT-Topic des Sensors
- Value – gemessener Wert
- TimeStamp – Zeitpunkt der Messung (UTC)

4.3 TopicLog

Log-Einträge im MQTT-Dashboard:

- Id – eindeutige GUID | TimeStamp – Zeitstempel
 - LogTyp – "INFO" oder "MQTT" | Direction – "IN"
 - Topic – MQTT-Topic (oder "-" bei INFO-Meldungen) | Payload – Inhalt
-

5. MQTT-Konfiguration

5.1 Verbindungsparameter

Parameter	Wert / Beschreibung
Broker-IP	Vom Benutzer einzugeben
Port	Standard: 1883 (max. 4 Stellen, nur Ziffern)
Client-ID	client1 (fest)
Subscribe-Topic	umweltmonitor/sensor/#
Pflanzentopic-Format	umweltmonitor/sensor/{PflanzenName}

5.2 Payload-Formate

Die Anwendung erkennt drei Formate automatisch:

Format	Beispiel	Felder
JSON	{"moistureSensor":500,"moisturePercent":65,"battery":80}	Alle drei
CSV	500,65,80	Sensor, Prozent, Batterie
Einzelwert	500	Nur Sensorwert

6. Funktionen der Benutzeroberfläche

6.1 Tab: Verbindung (MQTT-Dashboard)

Verbindungsaufbau

- IP-Adresse und Port eingeben; Enter-Taste startet die Verbindung
- Port: nur Ziffern erlaubt, max. 4 Stellen, Standard: 1883
- Bei leerem IP-Feld: Fehlermeldung im Log

Status-Anzeige

- Icon: grün (verbunden) / rot (getrennt) | UpTime: hh:mm:ss
- Broker: verbundene IP:Port | Empfangene Nachrichten | Fehleranzahl

Log-Tabelle

- Spalten: Zeitstempel, Typ (INFO/MQTT), Richtung, Topic, Payload
- Max. 500 Einträge; Clear-Button leert Liste und setzt Zähler zurück
- Logs werden in logs.json gespeichert und beim Start wiederhergestellt

6.2 Tab: Pflanzen

Pflanze hinzufügen

- Name eingeben + Hinzufügen klicken oder Enter drücken
- Duplikat-Prüfung mit Fehlermeldung; Name = MQTT-Topic-Segment

Statusfarben (Feuchtigkeit)

Farbe	Bereich	Status	Bedeutung
Rot	0% - 30%	Kritisch	Pflanze sofort giessen
Gelb	31% - 40%	Durstig	Feuchtigkeit sinkt
Gruen	41% - 70%	Perfekt	Optimale Feuchtigkeit
Blau	71% - 100%	Zu nass	Staunässe möglich

6.3 Tab: Statistiken

Diagramme

- Liniendiagramm: Feuchtigkeitsverlauf 24 h (X: HH:mm, Y: 0-100 %)
- Grenzklinien bei 30 % (orange) und 70 % (grün)
- Balkendiagramm: Vergleich aller Pflanzen (aktuelle Feuchtigkeit)
- PieCharts: Feuchtigkeits- und Temperatur-Halbkreis-Gauges

Handlungsempfehlung

Bereich	Status-Text	Empfehlung	Icon
0-30 %	Die Pflanze sofort giessen!	Kritisch!	AlertDecagram (Rot)
31-40 %	Die Feuchtigkeit sinkt!	Warnung!	Alert (Gelb)
41-70 %	(kein Eingriff noetig)	Perfekt!	Check (Gruen)
71-100 %	Die Pflanze ist zu nass (Staunässe)	Gefahr!	WaterAlert (Blau)

7. Datenpersistenz

Datei / DB	Format	Inhalt
plants.json	JSON	Pflanzen mit Sensorwerten; wird bei jeder Änderung gespeichert
logs.json	JSON	Log-Einträge (max. 500) + Fehlerzähler; beim Start geladen
SQLite-Datenbank	SQLite	Historische Sensordaten (Topic, Value, TimeStamp)

Alle JSON-Schreibvorgänge laufen in einem Hintergrund-Task mit Lock-Objekt, um Race Conditions bei gleichzeitigen MQTT-Nachrichten zu vermeiden.

8. Value Converter

Converter	Funktion
ValueToRangeConverter	int -> Low / Mid / High / TooHigh (Statuspunkte Pflanzentabelle)
ValueToRangeConverter2	int -> kritisch / durstig / optimal / zunass (Handlungsempfehlung)
BoolToColor	bool (IsConnected) -> Brushes.Green / Red (Status-Icon)
BoolToIconConverter	bool (IsConnected) -> SignalVariant / SignalOff (Icon-Art)

9. Bekannte Einschränkungen

- Temperatur-PieChart: wird noch nicht aus MQTT-Daten befüllt (Topic ausstehend)
- Kein Username / Passwort für MQTT-Broker (wird leer übergeben)
- Client-ID ist fest auf "client1" gesetzt
- plants.json und logs.json liegen im Arbeitsverzeichnis der Anwendung

10. Glossar

Begriff	Erklärung
MQTT	Message Queuing Telemetry Transport – leichtgewichtiges IoT-Nachrichtenprotokoll
Topic	Adressierungsschema in MQTT, z.B. umweltmonitor/sensor/Tomate
Payload	Nutzdaten einer MQTT-Nachricht (JSON, CSV oder Einzelwert)
Broker	MQTT-Server, der Nachrichten zwischen Clients vermittelt
MVVM	Model-View-ViewModel – Architekturmuster für WPF-Anwendungen
WPF	Windows Presentation Foundation – UI-Framework für Windows
Staunässe	Überschuss an Wasser im Boden, der Pflanzenwurzeln schädigen kann
DXA	Maßeinheit in Word-Dokumenten (1440 DXA = 1 Inch)

